САМАРСКИЙ УНИВЕРСИТЕТ
SAMARA UNIVERSITY

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего образования «Самарский национальный исследовательский университет имени академика С.П. Королева»

(Самарский университет)

Институт информатики и кибернетики

Кафедра киберфотоники

Отчёт

Лабораторная работа №2

«Высокопроизводительные вычисления в механике»

Выполнил: Свинцицкий З.

Группа 4446-010303D

Проверил: Неженский М.С.

«____» _____2026 г.

Оценка _____

Самара 2026

## Задание

**2)** Исследовать модификацию программы из практического задания №1 для параллельной работы по технологии OpenMP.

matMulOMP.cpp – исходники прилагаются.

**Цель работы**: исследовать параллельную реализацию умножения матриц с использованием технологии OpenMP. Определить зависимость времени выполнения от размера матрицы и числа потоков. Вычислить ускорение и эффективность параллельной программы.

**Выполнение работы:**

Компьютер для расчетов оснащён 6-ядерным процессором (6 физических ядер и 12 логических). Установка числа потоков OpenMP больше 12 не имеет практического смысла, поскольку большее количество потоков не будет использоваться эффективно.

Для корректного выполнения работы нужно исправить ошибки в исходном коде matMulOMP.cpp:

1) Поменять #pragma for на #pragma omp for
2) Добавить аргумент <threads> для удобного тестирования
3) Вынести до параллельного блока <omp_set_num_threads(THREADS)>
4) Убрать <clock()> - некорректен для многопоточного кода

```
    omp_set_num_threads(THREADS);
    double wtime = omp_get_wtime();

    int actual_threads = 0;

    #pragma omp parallel shared(actual_threads)
    {
        #pragma omp single
        actual_threads = omp_get_num_threads();

        #pragma omp for
        for (int i=0; i < N; i++)
            for (int j = 0; j < N; j++)
            {
                C[i*N + j] = 0;
                for (int k = 0; k < N; k++)
                    C[i*N + j] += A[i*N + k] * B[k*N + j];
            }
    }

    wtime = omp_get_wtime() - wtime;
```

Рис 1 – исправленная часть кода

Произведем расчеты используя исправленный код. Компиляция выполняется с помощью команд: g++ matMulSeq.cpp -o matMulSeq (из практического 1), g++ -fopenmp matMulOMP.cpp -o matMulOMP

```
PS C:\Users\zohan> cd C:\Users\zohan\Desktop
PS C:\Users\zohan\Desktop> g++ matMulSeq.cpp -o matMulSeq
PS C:\Users\zohan\Desktop> g++ -fopenmp matMulOMP.cpp -o matMulOMP
```

Рис 2 – компиляция программы

Ниже представлены результаты (при фиксированном числе потоков = 6):

```
PS C:\Users\zohan\Desktop> Get-Date; .\matMulSeq 100

1 апреля 2026 г. 19:35:48
Begin initializing ...
Begin calculating ...

Calculation time: 0.003 seconds


PS C:\Users\zohan\Desktop> Get-Date; .\matMulOMP 100 6

1 апреля 2026 г. 19:35:56
Matrix size: 100x100
Threads: 6
Begin calculating ...
Num of threads: 6
Elapsed omp time: 0.000999928 seconds


PS C:\Users\zohan\Desktop> Get-Date; .\matMulSeq 200

1 апреля 2026 г. 19:36:42
Begin initializing ...
Begin calculating ...

Calculation time: 0.026 seconds


PS C:\Users\zohan\Desktop> Get-Date; .\matMulOMP 200 6

1 апреля 2026 г. 19:36:48
Matrix size: 200x200
Threads: 6
Begin calculating ...
Num of threads: 6
Elapsed omp time: 0.00999999 seconds
```

Рис 3 – полученные результаты

```
PS C:\Users\zohan\Desktop> Get-Date; .\matMulSeq 300

1 апреля 2026 г. 19:37:14
Begin initializing ...
Begin calculating ...

Calculation time: 0.094 seconds


PS C:\Users\zohan\Desktop> Get-Date; .\matMulOMP 300 6

1 апреля 2026 г. 19:37:21
Matrix size: 300x300
Threads: 6
Begin calculating ...
Num of threads: 6
Elapsed omp time: 0.017 seconds


PS C:\Users\zohan\Desktop> Get-Date; .\matMulSeq 400

1 апреля 2026 г. 19:37:52
Begin initializing ...
Begin calculating ...

Calculation time: 0.219 seconds


PS C:\Users\zohan\Desktop> Get-Date; .\matMulOMP 400 6

1 апреля 2026 г. 19:38:00
Matrix size: 400x400
Threads: 6
Begin calculating ...
Num of threads: 6
Elapsed omp time: 0.0469999 seconds
```

Рис 4 – полученные результаты

```
PS C:\Users\zohan\Desktop> Get-Date; .\matMulSeq 500

1 апреля 2026 г. 19:38:22
Begin initializing ...
Begin calculating ...

Calculation time: 0.419 seconds


PS C:\Users\zohan\Desktop> Get-Date; .\matMulOMP 500 6

1 апреля 2026 г. 19:38:28
Matrix size: 500x500
Threads: 6
Begin calculating ...
Num of threads: 6
Elapsed omp time: 0.0899999 seconds


PS C:\Users\zohan\Desktop> Get-Date; .\matMulSeq 600

1 апреля 2026 г. 19:38:35
Begin initializing ...
Begin calculating ...

Calculation time: 0.747 seconds


PS C:\Users\zohan\Desktop> Get-Date; .\matMulOMP 600 6

1 апреля 2026 г. 19:38:45
Matrix size: 600x600
Threads: 6
Begin calculating ...
Num of threads: 6
Elapsed omp time: 0.144 seconds
```

Рис 5 – полученные результаты

```
PS C:\Users\zohan\Desktop> Get-Date; .\matMulSeq 700

1 апреля 2026 г. 19:39:11
Begin initializing ...
Begin calculating ...

Calculation time: 1.172 seconds


PS C:\Users\zohan\Desktop> Get-Date; .\matMulOMP 700 6

1 апреля 2026 г. 19:39:22
Matrix size: 700x700
Threads: 6
Begin calculating ...
Num of threads: 6
Elapsed omp time: 0.235 seconds
```

Рис 6 – полученные результаты

По полученным результатам построим сравнительную таблицу и проведем её анализ. Также вычислим ускорение и эффективность, которые вычисляются по формулам:

$$S = \frac{T_{seq}}{T_{omp}} \ ;$$

$$E = \frac{S}{p} \ , p = 6$$

| N | T_seq | T_omp | S | E |
|---|---|---|---|---|
| 100 | 0.003 | 0.0009 | 3.33 | 0.55 |
| 200 | 0.026 | 0.010 | 2.60 | 0.43 |
| 300 | 0.094 | 0.017 | 5.53 | 0.92 |
| 400 | 0.219 | 0.047 | 4.66 | 0.78 |
| 500 | 0.419 | 0.089 | 4.71 | 0.79 |
| 600 | 0.747 | 0.144 | 5.19 | 0.87 |
| 700 | 1.172 | 0.235 | 4.99 | 0.83 |

Таблица 1 – результаты

Построю график и сравним результаты T_seq и T_omp:

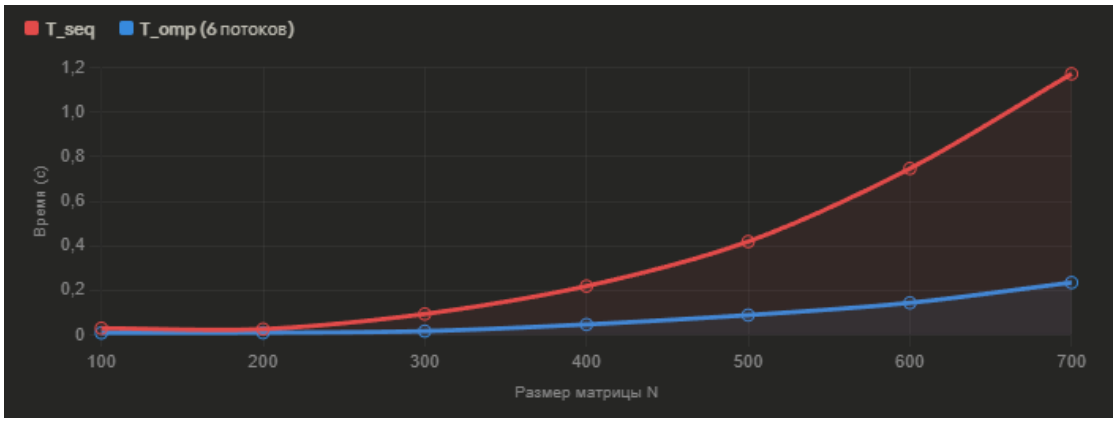


Рис 7 – график

Построю графики зависимости размерности матрицы N от ускорения S и эффективности E:

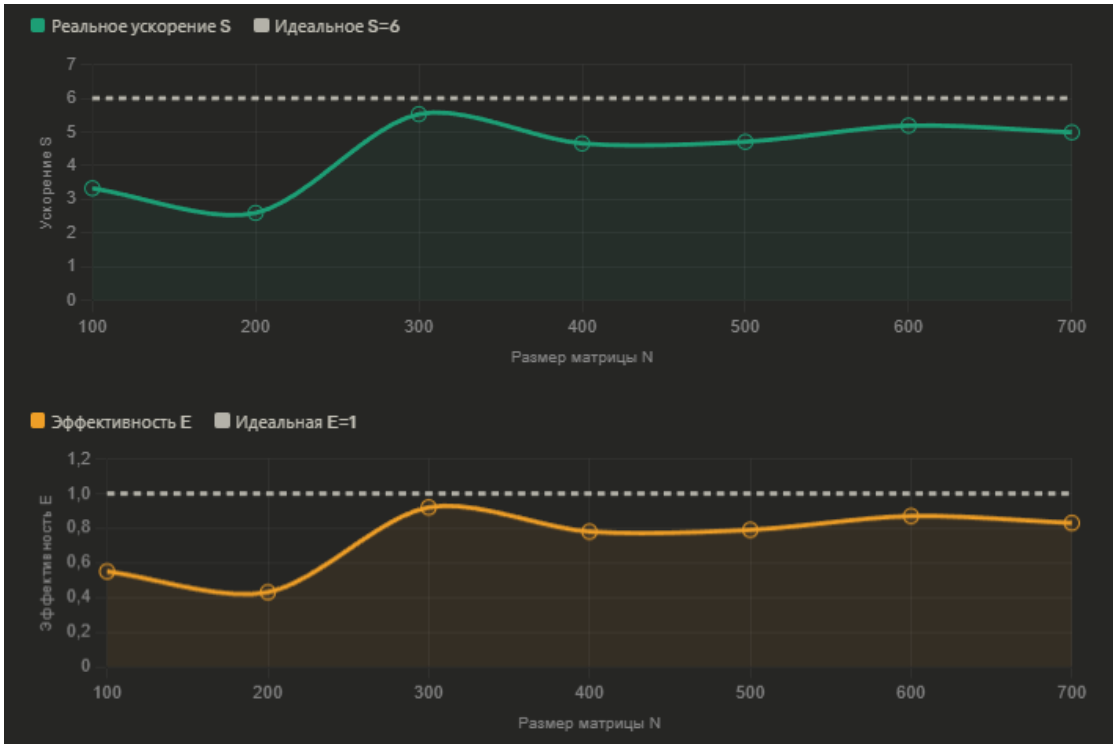


Рис 8 – график

Далее, фиксируем размерность матрицы (N=700), меняя число потоков от 1 до 12. Так как процессор имеет 6 физических ядер и 12 логических. Тестируем оба предела и получаем следующие результаты:

```
PS C:\Users\zohan\Desktop> Get-Date; .\matMulOMP 700 1

1 апреля 2026 г. 20:52:29
Matrix size: 700x700
Threads: 1
Begin calculating ...
Num of threads: 1
Elapsed omp time: 1.181 seconds


PS C:\Users\zohan\Desktop> Get-Date; .\matMulOMP 700 2

1 апреля 2026 г. 20:52:37
Matrix size: 700x700
Threads: 2
Begin calculating ...
Num of threads: 2
Elapsed omp time: 0.596 seconds


PS C:\Users\zohan\Desktop> Get-Date; .\matMulOMP 700 3

1 апреля 2026 г. 20:52:41
Matrix size: 700x700
Threads: 3
Begin calculating ...
Num of threads: 3
Elapsed omp time: 0.405 seconds


PS C:\Users\zohan\Desktop> Get-Date; .\matMulOMP 700 4

1 апреля 2026 г. 20:52:47
Matrix size: 700x700
Threads: 4
Begin calculating ...
Num of threads: 4
Elapsed omp time: 0.312 seconds
```

Рис 9 – полученные результаты

```
PS C:\Users\zohan\Desktop> Get-Date; .\matMulOMP 700 5

1 апреля 2026 г. 20:52:51
Matrix size: 700x700
Threads: 5
Begin calculating ...
Num of threads: 5
Elapsed omp time: 0.251 seconds


PS C:\Users\zohan\Desktop> Get-Date; .\matMulOMP 700 6

1 апреля 2026 г. 20:52:55
Matrix size: 700x700
Threads: 6
Begin calculating ...
Num of threads: 6
Elapsed omp time: 0.233 seconds


PS C:\Users\zohan\Desktop> Get-Date; .\matMulOMP 700 8

1 апреля 2026 г. 20:53:07
Matrix size: 700x700
Threads: 8
Begin calculating ...
Num of threads: 8
Elapsed omp time: 0.222 seconds


PS C:\Users\zohan\Desktop> Get-Date; .\matMulOMP 700 10

1 апреля 2026 г. 20:53:12
Matrix size: 700x700
Threads: 10
Begin calculating ...
Num of threads: 10
Elapsed omp time: 0.198 seconds
```

Рис 10 – полученные результаты

```
PS C:\Users\zohan\Desktop> Get-Date; .\matMulOMP 700 12

1 апреля 2026 г. 20:53:17
Matrix size: 700x700
Threads: 12
Begin calculating ...
Num of threads: 12
Elapsed omp time: 0.176 seconds
```

Рис 11 – полученные результаты

По полученным результатам построим сравнительную таблицу и проведем её анализ. Также вычислим ускорение и эффективность, которые вычисляются по формулам:

$$S(p) = \frac{T_{1\ \text{поток}}}{T_{omp}}\ ;$$

$$E(p) = \frac{S(p)}{p}, p = \text{поток}$$

| Число потоков (p) | T_omp | S(p) | E(p) |
|---|---|---|---|
| 1 | 1.181 | 1.00 | 1.00 |
| 2 | 0.596 | 1.98 | 0.99 |
| 3 | 0.405 | 2.92 | 0.97 |
| 4 | 0.312 | 3.79 | 0.95 |
| 5 | 0.251 | 4.71 | 0.94 |
| 6 | 0.233 | 5.07 | 0.85 |
| 8 | 0.222 | 5.32 | 0.67 |
| 10 | 0.198 | 5.96 | 0.60 |
| 12 | 0.176 | 6.71 | 0.56 |

Таблица 2 – результаты

Построю графики зависимости числа потоков от ускорения S и эффективности E, так же на графике показано идеальное значение S и E:

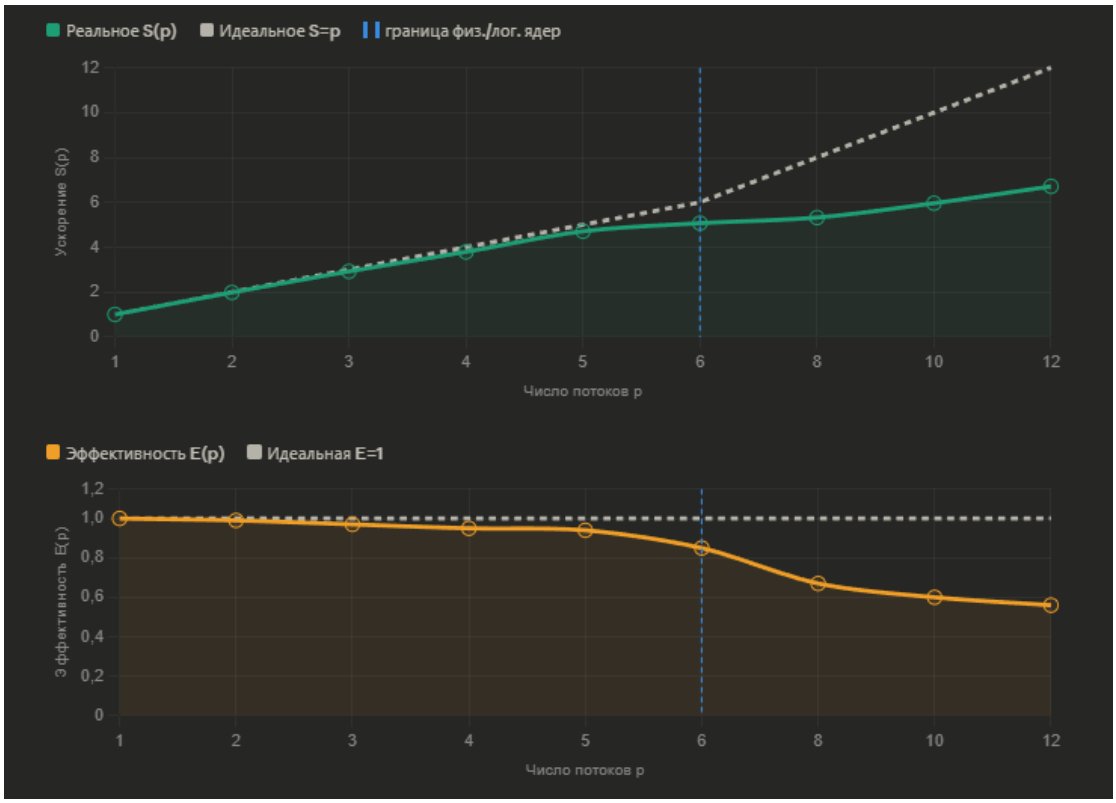


Рис 12 – график

**https://github.com/zohansen/matmulomp-lab2.git**

**Вывод:**

В данной лабораторной работе была исследована параллельная программа умножения матриц с использованием технологии OpenMP. Были проведены эксперименты для разных размеров матриц (100…700) и выполнено сравнение с последовательной версией.

Результаты показали, что параллельное выполнение значительно уменьшает время работы программы. С увеличением размера матриц эффект от распараллеливания становится более заметным, так как OpenMP позволяет эффективно ускорять вычисления, особенно для задач с большим объёмом данных.

Так же можно сделать вывод о том, что оптимальным с точки зрения соотношения ускорения и эффективности является использование p = 6 потоков, соответствующих числу физических ядер процессора. Дальнейшее увеличение числа потоков даёт прирост производительности, но ценой значительного снижения эффективности использования каждого потока. Полученные результаты хорошо согласуются с законом Амдала и практическими ограничениями технологии Hyper-Threading.

**Приложение 1. Исправленный файл matMulOMP.cpp**

```
#include <cstdlib>   // для функции atoi (преобразование строки в число)
#include <iostream>  // для вывода в консоль (cout)
#include "time.h"    // подключено, но явно не используется
#include <omp.h>     // библиотека OpenMP для параллельных вычислений
using namespace std;
// Функция вывода матрицы (в данной работе не используется)
template <typename T>
int matrixOutput(T *Mat, int size, string name)
{
   cout << "\"" << name << "\" matrix output:" << endl;
   for (int i=0; i < size; i++)
   {
      for (int j = 0; j < size; j++)
         cout << Mat[i*size + j] << " "; // доступ к элементу как к одномерному
массиву
      cout << endl;
   }
   return 1;
}

int main(int argc, char *argv[])
{
   // Проверка количества аргументов командной строки
   if (argc != 3)
   {
      cout << "Program usage: " << endl;
      cout << "./" << argv[0] << " <n> <threads>" << endl;
      cout << "where <n> is the size of square matrix, <threads> is number of threads"
<< endl;
      return -1;
   }

   // Получение параметров: размер матрицы и число потоков
```

```
const int N       = atoi(argv[1]);   // размер матрицы N x N
const int THREADS = atoi(argv[2]);   // количество потоков

cout << "Matrix size: " << N << "x" << N << endl;
cout << "Threads: " << THREADS << endl;

// Выделение памяти под матрицы A и B
float *A = new float[N*N];
float *B = new float[N*N];

// Инициализация матриц значениями
for (int i=0; i < N; i++)
    for (int j = 0; j < N; j++)
    {
        A[i*N + j] = (i+1) * (j+1);        // элемент A(i,j)
        B[i*N + j] = (i+1) + 2*(j+1);      // элемент B(i,j)
    }

// Выделение памяти под результирующую матрицу
float *C = new float[N*N];

cout << "Begin calculating ..." << endl;

// Установка количества потоков OpenMP
omp_set_num_threads(THREADS);

// Начало измерения времени
double wtime = omp_get_wtime();

int actual_threads = 0; // переменная для хранения реального числа потоков

// Начало параллельной области
#pragma omp parallel shared(actual_threads)
{
```

```
        // Этот блок выполнится только одним потоком
        #pragma omp single
        actual_threads = omp_get_num_threads(); // получаем фактическое число
потоков

        // Распараллеливание цикла
        #pragma omp for
        for (int i=0; i < N; i++)            // строки матрицы C
            for (int j = 0; j < N; j++)      // столбцы матрицы C
            {
                C[i*N + j] = 0;
                for (int k = 0; k < N; k++)    // вычисление скалярного произведения
                    C[i*N + j] += A[i*N + k] * B[k*N + j];
            }
    }
    // Конец параллельной области

    // Вычисление затраченного времени
    wtime = omp_get_wtime() - wtime;

    // Вывод результатов
    cout << "Num of threads: " << actual_threads << endl;
    cout << "Elapsed omp time: " << wtime << " seconds" << endl;

    // Освобождение памяти
    delete [] A;
    delete [] B;
    delete [] C;

    return 0;
}
```